

ICEBREAKER — INTERFAZE INTERVIEW TECHNICAL DOSSIER

A source-grounded study guide for the founding full stack engineer interview

Use pages 1–4 for the initial call. Use the remaining pages when an engineer asks how the product actually works, where the boundary lives, and what would have to change at SaaS scale.

MAIN CONCLUSION Icebreaker is strongest as evidence of AI orchestration, security boundaries, Linux execution, model reliability and end-to-end ownership. It is not evidence that this repository already implements Interfaze's Next.js, TypeScript, Postgres or multi-tenant API stack.

1 The Product in 60 Seconds

What it is

Icebreaker is an Ubuntu-based operating-system distribution that lets a user describe a system task in natural language. A Python Controller turns the request into a structured intent, applies policy and human approval, asks a restricted local model for an allowed tool call, and executes that call through a Rust daemon confined by Linux security primitives.

Who uses it and what problem it solves

The intended users are developers, operators and less technical Linux users who need system information or controlled changes without hand-writing shell commands. The hard problem is not text generation. It is giving probabilistic software a narrow, auditable path to privileged effects without handing a general model an unrestricted shell.

Core workflow

- The user types a request in the GTK chatbot, Textual terminal or CLI.
- A Quarantined Brain produces a JSON Intent Object but receives no execution tools.
- The Controller validates, raises risk when required, previews destructive changes and gates approval.
- The Privileged Brain produces a tool call constrained to the selected tool and schema.
- Rust mcpd validates again, applies filesystem and syscall controls, executes, and audits the result.

30 second interview explanation

I built Icebreaker as an AI-native Ubuntu distribution where natural-language requests can become real system operations without giving one general-purpose model a root shell. The system separates understanding from execution: a quarantined model creates a structured intent, a trusted Controller validates and risk-scores it, and a restricted local model can invoke only schema-defined tools through a Rust daemon protected by Landlock, seccomp, COW previews and tamper-evident audit logs. The relevant founding-engineer work was crossing product UX, model orchestration, Linux security, Python services, Rust execution and release engineering.

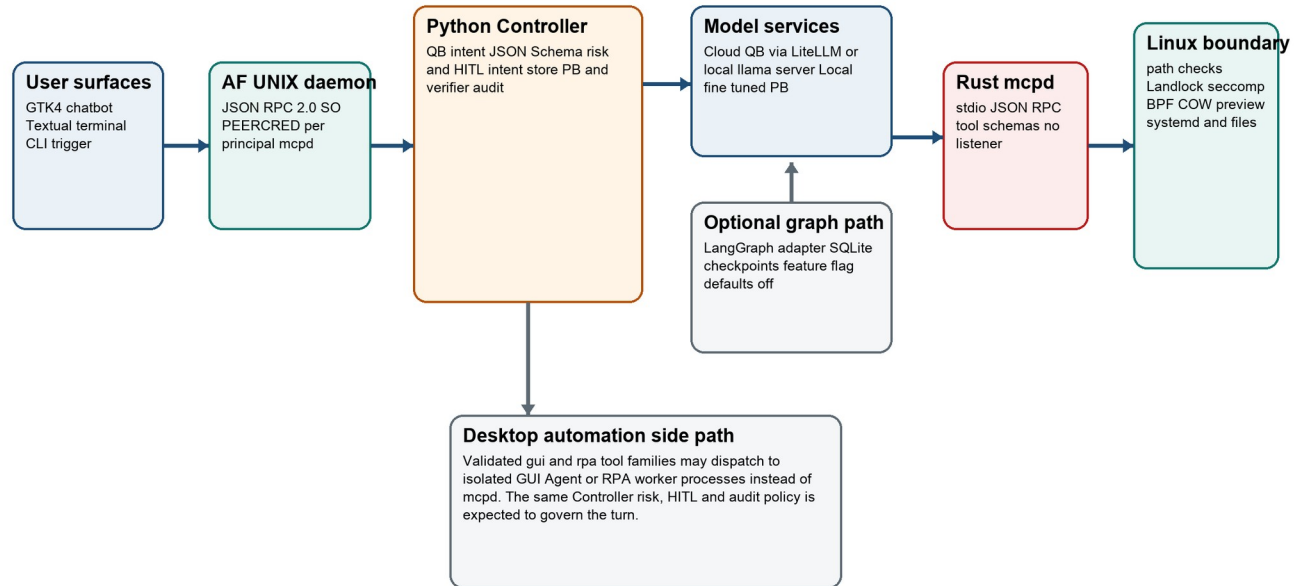
IMPLEMENTATION EVIDENCE `README.md`; `dual-brain/controller/main.py::Controller`;
`src/mcpd/src/server.rs::run_stdio_server`

LEVEL 1 INITIAL CALL

90 second technical explanation

I designed Icebreaker around a dual-brain boundary. The first model can read the user's request but has no MCP connections. It emits a strict Intent Object. The Python Controller normalizes server-owned fields, validates the object with JSON Schema, resolves paths to reduce TOCTOU risk, applies an escalate-only risk floor, optionally calls a reviewer, and presents high-risk work for approval. For destructive filesystem or package operations it asks mcpd for a real preview before approval. The intent is stored under a UUID, and the local fine-tuned Privileged Brain generates the permitted tool call. The Controller validates parameters against mcpd's advertised schema and a verifier checks alignment before dispatch. mcpd communicates over stdio, not a network listener, and adds path checks, Landlock, seccomp and a preview-bound commit ticket. The user sees streamed progress and the final result through an AF_UNIX daemon client. The distribution is assembled as amd64 and arm64 Ubuntu ISOs with Docker, systemd units, model checksums and CI gates.

Actual request path and trust boundaries



Declared boundary: PB should not see raw conversational text. Current code sends a structured PB envelope that includes the allowed tool, schema, target, content and hint; describe this carefully in interviews.

Figure 1 Actual architecture with the important trust boundaries

What makes it more than an LLM wrapper

- The model never receives a free-form shell tool. Tool names and arguments come from schemas.
- Risk cannot be lowered by a model; rules establish the minimum oversight.
- A destructive request is previewed and bound to a one-time commit ticket.
- The execution daemon is separately written in Rust and confined by the kernel.
- Every accepted, rejected and failed intent is intended to leave audit evidence.

LEVEL 1 INITIAL CALL

The Hardest Technical Problem

The hardest problem was preserving useful natural-language interaction while preventing model text from becoming authority. The Controller has to accept uncertainty at the language boundary, then progressively replace it with structured facts: an allowed action, resolved target, risk floor, dry-run evidence and a schema-valid tool call. A good answer should emphasize that safety is a protocol across components, not one prompt.

A Decision and Tradeoff to Defend

Decision: use two models separated by a trusted Controller. Benefit: external or document content can be handled by a model with no tools, while execution is narrowed to local policy and schemas. Cost: latency, duplicated prompts, more failure states, and a subtle invariant to maintain. The current PB envelope includes structured target, content and hint fields in addition to the UUID; therefore say that PB does not receive raw conversational input. Do not claim it literally receives only a UUID unless that drift is repaired.

IMPLEMENTATION EVIDENCE `dual-brain/controller/session.py::build_pb_user_turn; docs/2026-08-03_whitepaper_vs_reality_audit.md`

How Users Interact

Surface	Implementation	What the user sees
Chatbot	GTK4 and LibAdwaita	Messages, streamed tokens, reasoning steps and approval prompts
AI terminal	Textual	Shell and natural-language modes with progress and execution panels
CLI	Python client	A request or pre-formed intent sent to the daemon
Desktop automation	AT-SPI app APIs and optional RPA	GUI actions routed through dedicated worker processes

Scale and Performance Evidence

Production users, daily request volume and concurrency are not determinable from the repository. The measurable engineering surface is substantial: 4,654 Python test functions, 259 Rust test attributes, 25 mcpd JSON schemas, about 23,000 non-test Controller Python lines and about 7,000 mcpd Rust lines in the current workspace. These are codebase scale indicators, not customer adoption metrics.

Why Interfaze Should Care

- **DIRECT MATCH** Python AI infrastructure, model providers, structured output, retries, Docker and cloud deployment work.
- **TRANSFERABLE** High-volume API reliability maps to the same timeout, idempotency, backpressure and observability concerns, even though this product is local-first.
- **GAP** Next.js, React, Node.js, TypeScript, Postgres, Supabase, Fly.io and Stripe are not implemented as the product stack here.

LEVEL 1 INITIAL CALL

Initial Call Question Drill

Question	Answer anchor
Why did you build it?	To explore whether natural language could become a safe OS control surface, with execution authority separated from language understanding.
What did you personally build?	Describe end-to-end areas you can defend: Controller, model boundary, mcpd contract, UX clients, tests and ISO pipeline. Do not use Git history as proof of authorship.
What was hardest?	Turning model output into progressively stronger structured facts while maintaining usability and failure visibility.
What tradeoff did you make?	Two brains and repeated validation increase latency and complexity in exchange for narrower blast radius.
How does it fail?	Provider failures, invalid structured output, HITL timeouts, sandbox incompatibility and daemon crashes fail closed and should be audited.
How is this relevant?	It demonstrates model routing, provider abstraction, structured outputs, inference deployment, observability, defensive execution and full-stack product ownership.

Claims to Keep Precise

SAFE Python/Rust architecture, schema validation, mcpd stdio boundary, Linux sandbox, COW preview flow, audit chain and multi-arch packaging.

REVIEW Exact PB envelope, AgentGraph flag behavior, GUI/RPA dispatch differences, fallback trigger rules and the difference between controller audit and mcpd audit.

DANGER Claiming production traffic, a public SaaS API, Postgres, a React frontend, strict UUID-only PB input, or proven sub-100 ms end-to-end latency.

Five Files to Open Before the Call

- dual-brain/controller/main.py — the orchestration path
- dual-brain/controller/session.py — the actual PB envelope
- src/mcpd/src/server.rs — request dispatch and protocol boundary
- src/mcpd/src/tools/cow.rs — preview, ticket and commit model
- cx-distro/build.sh — how the product becomes a bootable distribution

If the call stays at product level, stop here. The remaining pages are the technical follow-up reference.

LEVEL 2 TECHNICAL FOLLOW UP

2 End to End Architecture

The public-facing boundary is not an HTTP API. A local client opens the Controller's Unix socket and sends newline-delimited JSON-RPC. The daemon authenticates the operating-system peer with `SO_PEERCRECRED`, creates isolated per-principal execution state, and streams notifications during the turn. mcpd itself accepts JSON-RPC only on stdin and writes responses on stdout.

Real request trace delete a file outside the home directory

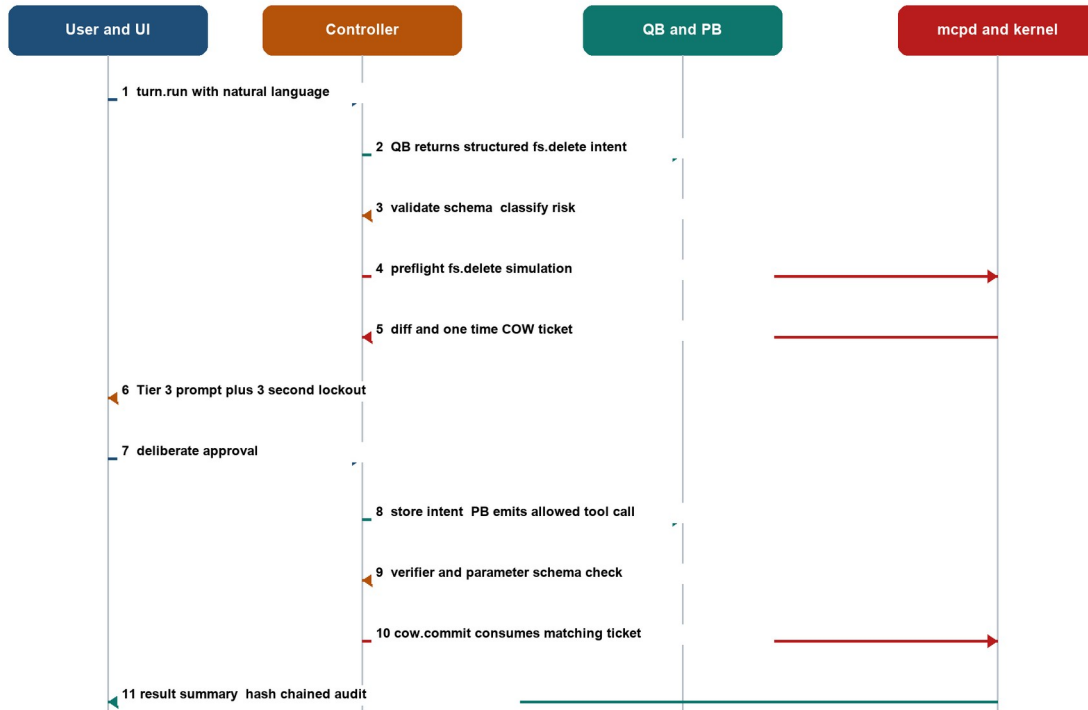


Figure 2 Destructive filesystem request with one preview and one approval

Component Responsibilities

Component	Technology	Responsibility
Client surfaces	GTK4, LibAdwaita, Textual	Capture requests, stream progress, render results and approvals
Daemon	Python AF_UNIX JSON-RPC	Peer auth, sessions, connection limits and event transport
Controller	Python	Schema, risk, HITL, model orchestration, dispatch and audit
QB and PB	LiteLLM or llama-server	Intent parsing and narrow tool-call generation
mcpd	Rust Tokio	Schema-validated system tools over stdio
Kernel boundary	Landlock and seccomp BPF	Filesystem and syscall confinement

IMPLEMENTATION EVIDENCE `dual-brain/controller/daemon.py::Daemon`; `controller/main.py::run_turn_streaming`; `src/mcpd/src/server.rs`

3 Tech Stack What We Actually Used

Layer	Technology	Where and why	Interview point
Frontend	GTK4 LibAdwaita Textual	gui/ and terminal/ local desktop clients	Event-driven UX without a browser stack
Backend	Python 3	Controller, providers, policy and daemon	Fast iteration around AI SDKs and schemas
Execution	Rust Tokio	mcpd system tools	Memory safety and explicit privileged boundary
AI	Gemini Anthropic OpenAI LiteLLM llama-server	QB provider registry and local PB	Provider abstraction and local inference
State	Memory SQLite JSONL	Sessions, graph checkpoints and audit	No product Postgres database
Infra	Docker systemd Ubuntu ISO GCP	Build, service supervision and VM deploy	Persistent host rather than serverless
Security	JSON Schema Landlock seccomp COW	Every execution layer	Defense in depth, fail closed

4 Three Most Important Technical Systems

Structured AI orchestration

Problem: translate untrusted language into a bounded effect. The Controller normalizes model-owned fields, validates the Intent Object, sets a rule-based risk floor, consults optional reviewer and trust policies, previews COW operations, gates HITL, invokes PB, validates the resulting tool parameters, dispatches and audits. The optional AgentGraph adapter expresses similar states as graph nodes but remains behind configuration.

Alternatives include one tool-using model or a deterministic parser. One model is simpler but combines perception and authority. A parser is safer but covers less language. At scale, isolate each stage behind typed messages, idempotency keys and queues while keeping the same policy order.

IMPLEMENTATION EVIDENCE `controller/main.py; intent_schema.py; risk_classifier.py; fallback_backend.py; agent_graph.py`

Sandboxed execution

mcpd exposes 24 static catalogue entries plus manifest-loaded tools in the current code path. Each method has a JSON Schema; dispatch validates before calling tool code. Destructive work returns a preview and a one-time intent ID. Commit verifies operation and subject before consuming the ticket. Landlock and seccomp add kernel enforcement beyond Python policy.

IMPLEMENTATION EVIDENCE `src/mcpd/src/tools/mod.rs; schema.rs; tools/cow.rs; sandbox/`

Model and distribution lifecycle

The local PB is trained and evaluated through SFT and DPO scripts, fused and quantized to GGUF, served by llama-server, pinned with SHA-256 and packaged into a multi-architecture ISO. Cloud QB providers share a registry and LiteLLM transport. This is valuable because model choice, weights, runtime, configuration and release artifacts are handled as one operating system product.

5 AI and Model Architecture

What this repository currently does

- QB providers: Gemini, Anthropic, OpenAI or local llama.cpp, selected through configuration and a backend registry.
- FallbackChain changes provider only for BrainProviderError. It does not hide schema, truncation or configuration errors.
- PB: a local fine-tuned Qwen 2.5 Coder 1.5B GGUF. Generic GBNF constrains the JSON tool-call shape; per-tool JSON Schema still validates semantics.
- PbRetryLoop performs bounded correction and can consult QB after repeated PB failures.
- Prompts live under controller/prompts and receive the generated tool catalogue. G25-style drift checks keep prompts and tools aligned.
- LangGraph is optional. It stores checkpoints in SQLite and uses an allowlist when deserializing checkpoint data.
- No embedding store or RAG pipeline is part of the core product. OCR and general multimodal document extraction are not implemented here.

Reliability answers

Condition	Current behavior	Production lesson
Provider timeout or 5xx	Wrapped as provider error; configured fallback may run	Bound retries and carry a deadline across providers
429 quota	Provider error path and possible fallback	Add per-provider budgets, jitter and circuit breakers
Malformed intent	Retry then schema rejection and audit	Structured output must fail closed
Invalid PB tool call	Schema validation plus bounded PB retry	Constrain generation and still validate after generation
Long context	Backend can raise BrainTruncationError	Do not send a truncation problem to another provider blindly
Provider unavailable	Fallback only if configured	Expose degraded state to user and metrics

What I should know as an engineer

Structured generation reduces syntax errors; it does not prove that the chosen action is correct. Reliability therefore needs separate checks for parse validity, policy, tool alignment, execution outcome and user-visible explanation. At API scale, add request-level idempotency, provider concurrency pools, token and dollar budgets, evaluation datasets, shadow routing and per-model success metrics.

IMPLEMENTATION EVIDENCE `controller/backends/; fallback_backend.py; pb_retry.py; prompts/; agent_graph.py`

6 Backend and API Deep Dive

The backend is a long-running local daemon rather than a stateless web service. It authenticates a Unix peer, creates a session, accepts `turn.run` or `intent.run`, starts a worker, and sends `turn.progress`, `turn.cot`, `turn.token` and `turn.result` notifications. The design supports multiple local clients while keeping the models and `mcpd` lifecycle outside the UI process.

- Concurrency: connection and turn limits are server-owned; sessions and `IntentStore` mutations use locks.
- Timeouts and retries: provider and HITL timeouts are configurable; PB and verifier retries are bounded.
- Streaming: clients receive newline-delimited JSON-RPC notifications, not SSE or WebSockets.
- Rate and cost controls: configuration limits turns, tokens and estimated spend; breaches abort rather than silently drop.
- Caching: preset verification and model registry data are cached; there is no Redis response cache.
- Queues and workers: background threads and subprocess workers exist, but there is no durable distributed job queue.

If traffic increased 100 times tomorrow

The first limits would be local model throughput, cloud-provider quotas, per-host memory, a single daemon process, SQLite checkpoint contention and `fsync`-per-audit-line latency. The current architecture is intentionally per-machine. Scaling it as a hosted service requires a new control plane, not just a larger VM.

7 Database Deep Dive

There is no Postgres, Supabase, ORM, tenant schema or vector database. `SessionStore` is an `RLock`-protected in-memory dictionary. `AgentGraph` can persist checkpoints in SQLite and prunes old rows. The audit log is canonical JSONL opened with `O_APPEND` and `fsynced` per line, with a hash chain for tamper detection. Model files, TOML configuration and YAML manifests provide additional filesystem state.

Concern	Grounded answer
Most expensive query	Not determinable; there is no product query workload. Checkpoint pruning performs a timestamp DELETE.
Most important index	Not determinable from the repository. A hosted design would index tenant, status and creation time.
Concurrent writes	Session memory is locked; SQLite uses one connection with <code>check_same_thread</code> disabled; audit writes are serialized and append-only.
Consistency	A process crash can lose in-memory session state; audit durability is stronger because each line is <code>fsynced</code> .

IMPLEMENTATION EVIDENCE `controller/session_store.py`; `agent_graph.py::_build_checkpointner`; `audit.py::AuditLog`

8 Frontend Deep Dive

The main interfaces are native desktop and terminal applications, not Next.js or React. ChatbotWindow sends a request through GtkDaemonClient, which schedules callbacks with GLib.idle_add so network-worker events do not mutate GTK state from the wrong thread. The Textual app owns input routing, progress cards, execution output and HITL presentation. Both are thin clients over the same daemon protocol.

- State management is local widget and session state, not Redux or server components.
- Loading state comes from progress and chain-of-thought events; partial response text streams as token events.
- Errors are rendered from structured result outcomes. Approval defaults to deny on timeout or interruption.
- A frontend interviewer may ask about GTK thread safety, cancellation, event ordering, reconnect behavior, accessibility and how the same contract would map to React.

IMPLEMENTATION EVIDENCE `gui/chatbot/window.py::ChatbotWindow; gui/daemon_client.py::GtkDaemonClient;`
`terminal/app.py::AiTerminalApp`

9 Deployment and Infrastructure

A push does not automatically deploy to production through a hosted CI pipeline visible in this repository. The implemented release path builds and validates artifacts locally or on a build VM. Docker provides the reproducible build environment. build.sh performs preflight and checksum checks, compiles mcpd, builds pinned llama.cpp, creates the Python venv, assembles a chroot, then runs debootstrap, mksquashfs and xorriso. systemd units supervise the Controller, PB, QB and mcpd processes. deploy.sh can install the stack on a GCP VM with XFCE and XRDP.

Interfaze technology	Icebreaker reality	Transferable concept
Lambda	Persistent systemd services	Choose lifecycle from workload latency and state
Fly.io	GCP VM and bootable ISO	Packaging, health, rollout and process supervision
Docker	Build container	Pinned reproducible toolchains and artifact verification
Hugging Face	Training base models and GGUF sources	Model provenance and deployment format
AWS Azure	Not implemented as runtime targets	Cloud-neutral interfaces need real provider adapters and tests

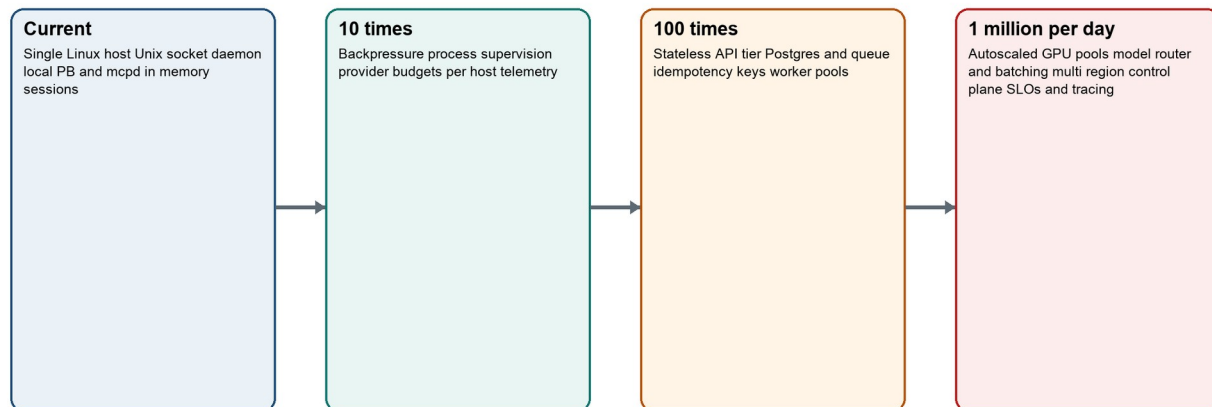
10 Reliability and Failure Modes

Failure	Current behavior	Better handling at scale
QB provider outage	Configured provider fallback on transport/API error	Circuit breaker and shared deadline
429 or quota	May fall back; user sees failure if chain exhausts	Budget-aware routing and jitter
Malformed intent	Schema retry or rejection and audit	Track error class by model version
PB violates schema	Bounded repair and QB consultation	Per-tool grammar and regression eval
HITL timeout	Deny	Persist resumable approval with expiry
mcpd crash	Turn fails; systemd can restart service	Health probe, retry only safe reads
Landlock unavailable	Refuse on Linux production path	Preflight kernel capability
Duplicate commit	COW ticket is consumed; replay rejected	Request idempotency across restarts
Audit write failure	Surfaced in system logging; execution evidence may be incomplete	Privilege-separated durable audit sink
Huge input	Schema and token limits	Ingress byte limits and admission control

11 Performance and Scale

The repository does not establish user counts, requests per second, GPU utilization under concurrency or production cost. It does establish explicit latency budgets and testable safeguards, but those are targets unless backed by current benchmark output. The main runtime costs are two or more model calls, local PB generation, verifier calls, fsync audit writes and any system operation itself.

Evolution path current product to one million requests per day



This is a redesign path, not a claim that the repository is already a multi tenant high volume API. The transferable skill is staged reliability engineering across model, policy and execution boundaries.

Figure 3 Honest scale evolution rather than a claim of existing SaaS scale

One million requests per day averages about 11.6 requests per second, but peak traffic, model duration and tokens matter more than the average. A 2-second inference at a 10-times peak can require hundreds of concurrent model slots. The answer should start with measured arrival rate, service time, token distribution and provider quotas, then size queues and GPU pools.

12 Security

Defense in depth each layer assumes the previous layer failed

1 Structured intent

QB has no tools. Controller rejects unknown fields and unsafe strings.

2 Policy and human gate

Rule based risk is a floor. Tier 3 requires preview and present tense approval.

3 Tool schema and path validation

The exact method and parameters are validated before dispatch.

4 Kernel confinement

Landlock limits filesystem reach. Secomp BPF limits the syscall surface.

5 COW ticket and audit evidence

Preview and commit are bound; append only hash chained logs make tampering detectable.

Failure should stop the operation and create evidence, not silently continue.

Figure 4 Security is a sequence of independent enforcement layers

Protection	Actual implementation	Limit or caveat
Authentication	SO_PEERCREC plus Unix socket ownership	Local OS identity, not web auth or OAuth
Authorization	Risk tiers, tool allowlists and filesystem roots	No SaaS RBAC or tenant isolation
Prompt injection	QB has no tools; structured boundary and verifier	Current PB envelope is broader than UUID-only prose
Command execution	No shell=True; explicit tools and args	Each new tool expands the trusted surface
Filesystem	Path checks, Landlock and COW	Linux-only enforcement; dev mode differs
Syscalls	seccomp deny and allow filters	Architecture-specific syscall maintenance
Secrets	Named environment references and child-env scrubbing	Cloud SDK and operator configuration remain sensitive
Audit	O_APPEND, fsync, redaction and hash chain	Local logs are not remote non-repudiation

Prompt Injection Answer

The key mitigation is capability separation. A model that reads untrusted text does not receive an execution tool. Its output becomes data that must survive schema validation, rule-based risk, an optional verifier and a second model constrained to one allowed tool. Kernel controls still assume every upstream layer can fail. The honest caveat is that structured fields such as content can carry attacker-controlled bytes; safety depends on treating them as data, never shell syntax.

Section 13 What I Personally Need to Be Able to Defend

Area	Explain	Likely follow up	Review
Controller turn	Why each stage is ordered this way	Could validation happen after PB?	controller/main.py
PB boundary	What PB actually receives	Is it really UUID only?	controller/session.py
Schemas	Intent versus tool parameter validation	What does JSON Schema not prove?	intent_schema.py and mcpd/schema.rs
Risk and HITL	Escalate-only floor and lockout	Can a model lower risk?	risk_classifier.py and hitl.py
COW	Preview ticket and commit binding	What prevents a target swap?	tools/cow.rs
Sandbox	Path checks, Landlock and seccomp order	Why all three?	sandbox/
Provider reliability	Fallback and PB retry boundaries	When should fallback not run?	fallback_backend.py and pb_retry.py
Release	Checksums, systemd and ISO stages	How do you stop wrong weights shipping?	cx-distro/build.sh

14 Interfaze Relevance Map

Project evidence	Interfaze fit	Transferable knowledge	What I should say
Python, AI APIs, prompts	DIRECT MATCH	Provider SDKs, prompt assembly, retries and local inference	I built orchestration and explicit failure boundaries around model calls.
Docker, GCP, cloud deploy	DIRECT MATCH	Build containers and a GCP VM deployment path	GCP is evidenced; AWS and Azure are not runtime targets here.
GPU inference, Hugging Face	DIRECT MATCH	Model acquisition, SFT/DPO, GGUF conversion and llama-server	I can discuss the lifecycle from model source to verified distributable weight.
Model routing, reliability	DIRECT MATCH	Registry, provider fallback, verifier and bounded repair	I separate transport failure, invalid output and policy rejection.
Structured output, prompt engineering	DIRECT MATCH	JSON Schema, GBNF, catalogue-generated prompts and validation	Constrained syntax still needs semantic and policy checks.
Developer APIs	TRANSFERABLE	Typed JSON-RPC contract and schema catalogue, but no public HTTP API	The contract discipline transfers; the external API surface would be new.
Node.js, TypeScript	GAP	The product backend is Python and Rust	I would not present this repository as Node or TypeScript experience.
Next.js, React	GAP	Native event-driven UI concepts transfer	I know streaming state and UI-thread boundaries, but not from React here.
Postgres, SQL, Supabase	KNOW CONCEPTUALLY	SQLite checkpoints and append-only state only	A hosted control plane needs tenant data, pooling, indexes and migrations.
Serverless, Fly.io	TRANSFERABLE	Persistent service and worker lifecycle	Cold starts, local models and GPU affinity drive the hosting choice.
OCR, document extraction	KNOW CONCEPTUALLY	No OCR pipeline in the core product	I would separate ingest, extraction, confidence, review and retention.
Multimodal AI	KNOW CONCEPTUALLY	Vision grounding exists; general multimodal serving does not	The security boundary generalizes, but the modality pipeline is unbuilt.
High-volume scaling	TRANSFERABLE	Limits, backpressure concepts and per-host isolation	There is no traffic proof; I can design queues, SLOs and autoscaled pools.
Stripe	GAP	No payments implementation	I would not claim payments experience from this project.

15 Questions I Am Likely to Get About This Project

High probability initial call

- 1 What user problem made an AI-native operating system worth building?
- 2 Walk me through a request from the GTK chatbot to a real system effect.
- 3 Why use two models instead of one capable model with tools?
- 4 What was the hardest engineering failure you found and how did you stop it recurring?
- 5 Which parts did you own deeply enough to debug without an AI coding assistant?
- 6 How does this experience transfer to developer-facing AI infrastructure?

Technical follow up

- 7 How does the daemon authenticate clients without application login code?
- 8 Where is the Intent Object validated, and how are extra fields rejected?
- 9 Why does provider fallback run for transport errors but not schema errors?
- 10 What exactly happens between a Tier 3 preview and cow.commit?
- 11 Why validate the PB call again after using grammar-constrained decoding?
- 12 How do GTK and Textual receive streaming state from the same backend?
- 13 What survives a daemon restart and what is lost?
- 14 How are model weights and native binaries verified before release?

Deep dive and staff engineer questions

- 15 Which architectural claim in the whitepaper is weaker in the current code?
- 16 What race exists between path resolution, approval and execution, and which layers reduce it?
- 17 What happens when mcpd is killed after execution but before the response reaches the Controller?
- 18 How would you add multi-tenant authorization without weakening per-host kernel isolation?
- 19 Which SLOs and traces would you add before serving one million requests per day?
- 20 If you rewrote this as Node.js APIs plus Python GPU workers, where would the typed contract live?

Practice rule: answer with one design choice, one concrete code location, one failure mode and one tradeoff. That pattern demonstrates implementation knowledge without rambling.

Section 16 Rapid Fire Technical Cross Examination

Claim	Uncomfortable follow up
Dual-brain isolation	What crosses the boundary today? Why does session.py include target and content?
No invalid model output	Does generic GBNF enforce each tool's parameter schema, or only JSON shape?
COW safety	Is this overlays, a simulation, a snapshot or a ticket? What can be restored?
No network mcpd	How do you prove it has no listener, and how does the Controller launch it?
Tamper-evident audit	What attack does a hash chain detect, and what can a privileged local attacker still do?
Provider resilience	Which exception types trigger fallback? What happens after partial streaming?
Agent orchestration	Is LangGraph the production default? Which path remains canonical when the flag is off?
Multi-architecture release	Which bugs can QEMU emulation hide, and why was native arm64 verification required?
Thousands of tests	Which tests cross a real binary or kernel boundary rather than mocking collaborators?
Production ready	Where are the measured SLOs, user traffic, distributed tracing and rollback evidence?

17 Tradeoffs I Should Know

Decision	Benefit	Drawback	Alternative when growth demands it
Two brains	Smaller injection blast radius	More latency and boundary drift	Single model with capability broker
Rust mcpd	Narrow privileged surface	Cross-language complexity	Python worker in a stronger external sandbox
AF_UNIX daemon	OS-native auth and low overhead	Local-only interface	HTTP/gRPC control plane
Local PB	Privacy and predictable authority	Hardware and packaging cost	Hosted model with private networking
Manual ISO assembly	Works around Noble live-build breakage	More custom release code	Managed image pipeline when tooling supports it
fsync every audit line	Strong durability	Latency and storage pressure	Batching to a privilege-separated sink

18 What I Would Change Today

Class	Improvement
Reliability	Make one orchestration path canonical and retire feature-flag twins after parity tests.
Scale	Split a hosted control plane from per-host execution agents; add queueing, idempotency and backpressure.
Security	Restore a minimal PB envelope and move audit to a privilege-separated remote-capable sink.
Observability	Add OpenTelemetry traces and SLOs for every model, policy, HITL and tool stage.
Developer experience	Generate typed clients and contract tests from the intent and MCP schemas.
Cost and latency	Add token budgets, semantic caching for safe reads, model shadowing and measured routing policy.

19 The Files I Absolutely Must Review

File or module	Know this cold
controller/main.py	Canonical turn ordering and error behavior
controller/session.py	What the PB actually receives
controller/intent_schema.py and schemas/intent.json	Intent contract and metacharacter policy
controller/fallback_backend.py and pb_retry.py	Two different reliability loops
controller/daemon.py	Unix peer auth, sessions and streaming RPC
src/mcpd/src/server.rs	stdio protocol and dispatch
src/mcpd/src/tools/cow.rs	Preview evidence and commit binding
src/mcpd/src/sandbox/	Landlock and seccomp order
cx-distro/build.sh	Artifact supply chain and ISO stages

Knowledge Danger Zones

SAFE Independent validation layers. Review controller/main.py and mcpd/schema.rs. Know intent versus tool-schema validation. Question: why validate twice? Credible: each trust boundary rejects independently.

REVIEW PB envelope. Review controller/session.py and docs/2026-08-03_whitepaper_vs_reality_audit.md. Know structured fields versus raw conversation. Question: what reaches PB? Credible: name every field and the narrower guarantee.

REVIEW Legacy versus AgentGraph. Review controller/main.py, agent_graph.py and config.py. Know flag default and parity. Question: which path is canonical? Credible: name the default and remaining divergence risk.

DANGER Web SaaS claims. Review daemon.py, session_store.py and agent_graph.py. Know local peer auth, in-memory sessions and optional SQLite. Question: how would tenancy work? Credible: admit the gap and design a stateless control plane, Postgres, queues and per-host agents.

Final Priority List

IF I HAVE ONLY 30 MINUTES Read pages 1–4, rehearse the 30- and 90-second scripts, then inspect main.py, session.py and cow.rs.

IF I HAVE 2 HOURS Add the real request trace, fallback versus retry, daemon authentication, security layers, ten cross-examination claims and the InterfaZe relevance map.

IF I HAVE HALF A DAY Walk one turn in a debugger or tests, read the nine priority files, explain each failure mode aloud, and design the hosted control plane on a whiteboard.

20 Interview Cheat Sheet

PRODUCT

An Ubuntu-based operating system that turns natural-language requests into policy-gated, schema-valid and sandboxed system operations.

USER

Developers, operators and Linux users who want a safer natural-language control surface.

PROBLEM

A capable model can understand intent but should not receive unrestricted execution authority.

SCALE

Production traffic is not determinable. Current evidence: 4,654 Python test functions, 259 Rust test attributes, 25 tool schemas and multi-architecture ISO tooling.

STACK

Python, Rust, Tokio, JSON Schema, GTK4, Textual, AF_UNIX JSON-RPC, LiteLLM, Gemini, Anthropic, OpenAI, llama.cpp, LangGraph optional, SQLite, systemd, Docker and Ubuntu ISO tooling.

ARCHITECTURE

User → UI → Unix daemon → QB Intent Object → validation and risk → COW and HITL → PB tool call → verifier → mcpd → Landlock and seccomp → audit → response.

MY 3 STRONGEST TECHNICAL TALKING POINTS

- Structured AI orchestration with separate perception and execution roles.
- Rust execution boundary with schema, kernel sandbox and preview-bound commit.
- Model-to-ISO ownership across providers, local inference, checksums and multi-arch release.

HARDEST TECHNICAL PROBLEM

Replacing probabilistic language with progressively stronger facts without destroying usability: allowed action, resolved target, risk floor, dry-run evidence, human decision and validated tool arguments.

BIGGEST TRADEOFF

Two brains reduce the blast radius of prompt injection, but add latency, duplicate policy paths and a boundary that can drift unless tests inspect the exact payload.

FAILURE MODE I UNDERSTAND WELL

A destructive request can change between preview and execution. Icebreaker binds the preview to a one-time ticket and verifies operation and subject at commit; path and kernel checks still apply.

WHAT I WOULD REDESIGN

Make one orchestration path canonical, restore the smallest possible PB envelope, add distributed tracing, and separate a hosted control plane from per-host execution agents.

INTERFAZE CONNECTION

- Direct: Python AI services, structured outputs, provider routing, retries, Docker and model deployment.
- Transferable: reliability, developer UX and staged scaling.
- Gap: this repo is not Next.js, TypeScript, Postgres, Supabase or Stripe.

30-SECOND VERSION

I built Icebreaker as an AI-native Ubuntu distribution where natural language can trigger real system work without giving a general model a root shell. A quarantined model creates a structured intent, a trusted Controller validates and risk-scores it, and a restricted local model invokes schema-defined tools through a Rust daemon protected by Landlock, seccomp, COW previews and tamper-evident audit logs.

90-SECOND VERSION

The user starts in a GTK or Textual client connected to a Python daemon over an authenticated Unix socket. The QB turns language into a strict Intent Object. The Controller normalizes server fields, validates JSON Schema, resolves targets, applies an escalate-only risk floor, previews destructive work and gates HITL. It stores the intent and asks a local fine-tuned PB for the allowed tool call. A verifier and the tool's own schema check the result. Rust mcpd accepts only stdio JSON-RPC and executes behind path checks, Landlock, seccomp and a preview-bound COW ticket. Progress and results stream back to the UI, and the turn enters a hash-chained audit log. Docker, systemd, checksums and multi-arch ISO scripts turn the stack into a distributable OS.

3-MINUTE TECHNICAL VERSION

Start with the threat model: a model that reads arbitrary text can be prompt-injected, so perception and execution cannot share unrestricted capability. Icebreaker gives the QB no tools. Its only product is a schema-shaped intent. The Controller rejects unknown fields and unsafe values, establishes risk from deterministic rules, and can only raise oversight. Tier 3 work receives a real mcpd simulation before the user can approve it. The intent then enters an in-memory UUID store. The PB receives a structured envelope and must produce one allowed tool call; generic GBNF constrains syntax, while JSON Schema and a verifier check semantics. mcpd validates again and runs behind Landlock and seccomp. Destructive work uses a one-time ticket whose operation and subject are checked at cow.commit. The daemon streams events to GTK and Textual clients and records append-only, fsynced, hash-chained audit evidence. The honest limitations are equally important: the current repository is a per-host product, not a multi-tenant web API; LangGraph is optional; the PB envelope is broader than an opaque UUID; and production traffic is unknown. To reach Interfaze scale I would add a stateless authenticated control plane, Postgres, durable queues, idempotency, autoscaled GPU pools, model routing telemetry and SLO-driven operations while keeping the per-host execution boundary narrow.